Data Catalog v2 — Design Spec

Date: 2026-05-23
Supersedes: 2026-04-24-data-catalog-sync-design.md
Status: Implemented

What changed in v2

v1 established the catalog schema, alias-based reconciliation, and sync workflow. v2 adds:

1. **Structured risk thresholds** — `risk_threshold` / `risk_direction` fields on columns, replacing threshold info buried in description text.
2. **Programmatic threshold injection** — post-distillation step auto-injects threshold values into chart data without LLM extraction.
3. **Catalog access diagram** — documents all consumers and the format each sees.
4. **`DataCatalog.get_thresholds()`** — new method for threshold lookup.

---

§1 — Risk threshold metadata

Problem

Thresholds were embedded in description prose: *"Values below 721 are risky."* The distiller LLM had to parse this text, remember to include `threshold` keys on every data row, and get the value right. It frequently failed — charts rendered without threshold reference lines, or with incorrect values.

Solution

New optional fields per column in `config/data_profiles/*.yaml`:

```yaml
credit_loss_prob:
  dtype: float
  description: "ML model score predicting default. Scores from 10-100 are risky."
  risk_threshold: 10          # NEW — structured threshold value
  risk_direction: above       # NEW — "above" or "below"
```

38 columns across `model_scores.yaml`, `bureau.yaml`, and `spends.yaml` have been updated.

Catalog API addition

```
catalog.get_thresholds("model_scores")
# → {"credit_loss_prob": {"value": 10, "direction": "above"},
#    "tot_struct_risk_score": {"value": 20, "direction": "above"},
#    "times_30_dpd": {"value": 1, "direction": "above"}, ...}
```

`get_schema()` also includes `risk_threshold` and `risk_direction` in the per-column dict when present, so `get_table_schema` tool output surfaces them to specialist LLMs.

---

§2 — Post-distillation threshold injection

After the distiller produces KnowledgePoints with chart specs (`viz`), a programmatic step injects threshold values from the catalog into the `numbers` array:

```
Distiller output:
  numbers: [{"period": "2024-01", "credit_loss_prob": 0.8, "tot_struct_risk_score": 8.4}, ...]
```

```
  viz: {kind: "trend_dual", y_fields: ["credit_loss_prob", "tot_struct_risk_score"]}

After auto-injection:
  numbers: [{"period": "2024-01", "credit_loss_prob": 0.8, "tot_struct_risk_score": 8.4,
          "threshold_credit_loss_prob": 10, "threshold_tot_struct_risk_score": 20}, ...]
```

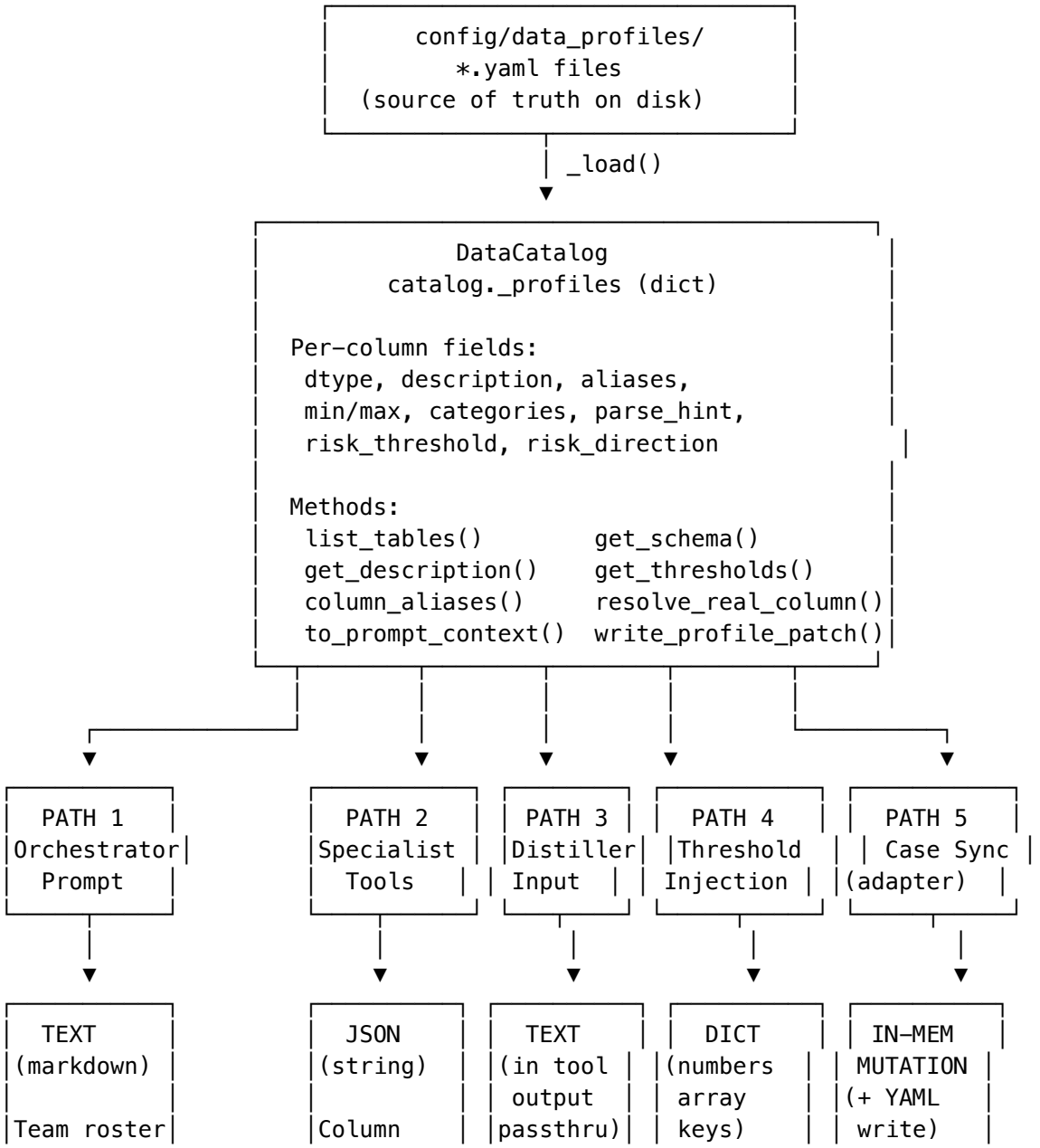
The renderer draws dashed reference lines at these values. No LLM involvement — thresholds come directly from structured profile metadata.

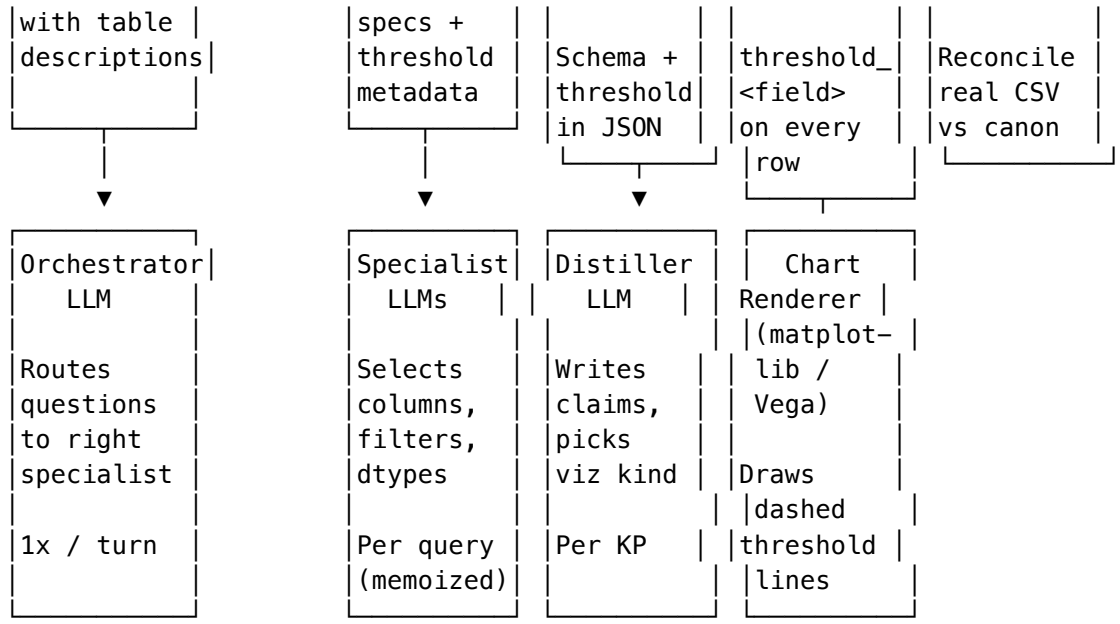
Guard: Injection only fires when no `threshold_<field>` key already exists on the rows (preserves any values the distiller explicitly set).

---

§3 — Catalog access architecture

Access diagram



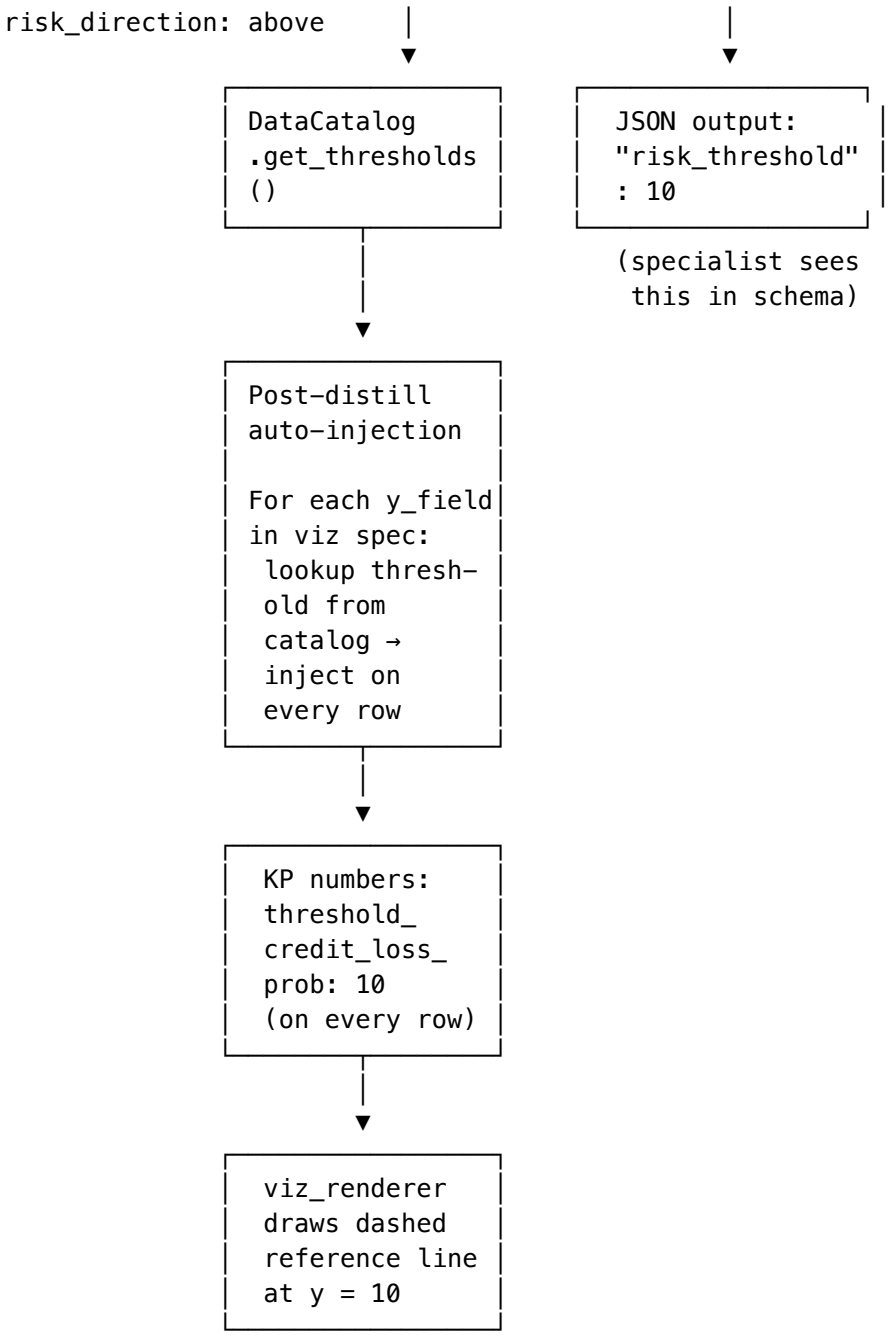


Access summary

Path	Consumer	Method	Format	Purpose	Frequency
1	Orchestrator LLM	`get_description()`	TEXT (markdown roster)	Route questions to the right specialist based on table ownership	1x per turn
2	Specialist LLMs	`get_table_schema()`, `list_available_tables()`	JSON string	Column selection, filter values, data types, threshold values	Per query (memoized)
3	Distiller LLM	Passthrough via raw tool outputs	TEXT (JSON in tool output)	Write KP claims, choose viz kind (thresholds now auto-injected, not LLM-parsed)	Per distillation
4	Chart renderer	`get_thresholds()` via `AppContext._catalog`	DICT of `numbers` array keys	Auto-inject `threshold_<field>` on every row for dashed reference lines	Per KP post-distillation
5	adapter.py	`_profiles` direct + `write_profile_patch()`	IN-MEMORY dict mutation	Reconcile case CSV columns vs canonical profiles at case open	1x per case

Threshold data flow (new in v2)

```
YAML profile                          Specialist tool call
risk_threshold: 10  ──────┐           get_table_schema()
```



---

§4 — Profile schema (cumulative)

All fields on a column entry, including v1 and v2 additions:

```yaml
column_name:
  # --- Core (v1) ---
  dtype: string | int | float | categorical | date
  description: "human-readable meaning"
  distribution: normal | poisson | uniform       # simulation
  mean: float                                    # simulation
  std: float                                     # simulation
  min: number                                    # simulation + range context
```

```yaml
  max: number                                  # simulation + range context
  categories: {value: probability, ...}        # categorical only
  aliases: [real_csv_name, ...]                # v1 sync
  description_pending: boolean                 # v1 sync (default false)
  parse_hint: "strptime format"                # v1 sync
  dtype_pending_review: boolean
  categories_pending_review: boolean

  # --- Risk threshold (v2) ---
  risk_threshold: number                       # the cutoff value
  risk_direction: above | below                # which side is risky
```

All new fields are optional with no defaults needed — backward compatible.

---

§5 — Columns with thresholds (38 total)

model_scores (30 columns)

Column	Threshold	Direction
credit_loss_prob	10	above
tot_struct_risk_score	20	above
times_30_dpd	1	above
last_cycle_cut_revolve_rate	0.46	below
cust_expsr_avg_rem_12m_ratio	3.15	above
tpf_internal_delinq_idx	5.8	above
delnqncy_ind_intrnl	0.5	above
avutil_exrvlv_balgt50	75	above
hcam_src_trnd_indx	-19	below
oop_interaction	28	above
hcam_bal_trnd_indx	2.65	above
cust_ext_delinq_idx	5	above
cust_eff_se_cdss_5_180_day_score	2	above
cust_lend_acct_paydown	0.1	below
cust_open_acct_paydown	0.3	below
cust_pymt_chan_risk_score	0.05	above
time_wtd_return_index	0.2	above
cust_rnn_score	0.028	above
cust_min_due_12mo_avg	0.08	above
gam_clr_erly_risk_score	785	below
cust_lexis_nexis_blended_score	693	below
cust_sbfe_score	863	below
cust_dnb_paydex_score	61	below
(+ 7 more with niche thresholds)		

bureau (7 columns)

Column	Threshold	Direction
fico_score	721	below
sbfe_score	863	below
ln_credit_score	681	below
ln_blended_score	694	below

Column	Threshold	Direction
css_score	71	below
fss_score	51	below
paydex_score	61	below

spends (4 columns)

Column	Threshold	Direction
merchant_risk_score	0.7	above
spend_concentration	2.4	above
rnn_spend_score	0.017	above
spend_divergence_index	1.0	below

---

§6 — Impact on distiller performance

Before v2	After v2
Distiller parses “Values from 10-100 are risky” from description text	Threshold is structured metadata — no parsing needed
Distiller must remember to include `threshold_<field>` on every row	Thresholds auto-injected post-distillation from catalog
Frequently failed □ charts without reference lines	Guaranteed correct — programmatic injection from source of truth
Distiller generated ~800+ tokens for numbers array with thresholds	Distiller generates fewer tokens; thresholds spliced in at zero LLM cost

---

§7 — Unchanged from v1

Everything in the original spec (§1-§8) remains in effect:

- Architecture and file layout
- Recognition algorithm (4-stage cascade)
- YAML catalog core schema (extended, not replaced)
- Reconciliation skill workflow
- Downstream consumer behavior
- pandas scoping
- Testing approach
- Open questions / deferred items